PulmoScan AI

PROJECT REPORT

Intelligent Pneumonia Detection Using CNN

A deep learning web application for chest X-ray classification with Grad-CAM explanations

SUBMITTED BY	Tanisha Dey
PROGRAMME	B.Tech, Data Science
DATE	November 2026
LIVE DEMO	pulmoscan-ai-topaz.vercel.app

Educational prototype. Not a medical device and not clinically validated.

Abstract

Pneumonia is a leading infectious cause of death in young children, and chest radiography is a standard tool for assessing it. This project, PulmoScan AI, is an end-to-end deep learning system that classifies a chest X-ray as *Normal* or *Pneumonia* and explains each prediction visually. A compact convolutional neural network (CNN) with 106,193 parameters was designed and trained from scratch in TensorFlow/Keras on the public Kermany et al. dataset of 5,856 paediatric chest X-rays. The model is served by a FastAPI backend that returns the predicted class, the probability of each class and a Grad-CAM heatmap, and is used through a responsive React web application.

On the official held-out test set of 624 images the model achieves **78.8% accuracy** and a **ROC-AUC of 0.930**. It is highly sensitive (98.5% recall for pneumonia, 6 of 390 cases missed) but has low specificity (46.2%): 126 of 234 normal X-rays are flagged as pneumonia. Validation accuracy is 95.2%; the gap to the test score is analysed in the report and attributed mainly to a distribution shift between the training pool and the official test set. All figures in this report are measured results produced by the project's evaluation script; none are estimated.

The system is deployed and publicly accessible: the frontend on Vercel and the model server as a Docker container on Render. The project is an educational prototype. It has not been clinically validated and must not be used for diagnosis.

Keywords: convolutional neural network, pneumonia detection, chest X-ray, Grad-CAM, explainable AI, TensorFlow, FastAPI, React.

Contents

Abstract . . . 2

Contents . . . 3

1. Introduction . . . 4

2. Background and Related Work . . . 4

3. Dataset and Preprocessing . . . 5

4. Model Architecture . . . 6

5. Training . . . 7

6. Results and Evaluation . . . 8

7. Explainability with Grad-CAM . . . 10

8. System Design and Implementation . . . 11

9. Deployment . . . 13

10. Limitations and Future Work . . . 13

11. Conclusion . . . 14

References . . . 15

1. Introduction

1.1 Background

Pneumonia is an infection that inflames the air sacs (alveoli) of the lungs, which may fill with fluid or pus. It is most often caused by bacteria or viruses. The World Health Organization identifies it as the largest single infectious cause of death in children worldwide. Pneumonia is usually treatable, and outcomes are considerably better when it is recognised and treated early.

On a chest X-ray, healthy lungs appear dark because air lets X-rays pass through. Infected regions hold fluid and appear as lighter, hazy patches known as opacities or consolidation. Interpreting these images requires training and time, which motivates research into computer-aided analysis.

1.2 Problem statement

Given a frontal chest X-ray image, predict whether it shows signs of pneumonia, report how confident the model is, and show which regions of the image influenced the prediction, through an application that a non-technical user can operate.

1.3 Objectives

- Build a complete, reproducible CNN training pipeline: preprocessing, augmentation, training with early stopping, and evaluation.
- Evaluate honestly on data the model has never seen, reporting accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix.
- Explain every prediction with Grad-CAM so the model is not a black box.
- Serve the model through a validated REST API and a polished, responsive web interface.
- Deploy the system so it is publicly usable, with the frontend and model server on appropriate hosts.

1.4 Scope

The system performs binary classification (Normal / Pneumonia) of paediatric chest X-rays. It does not distinguish bacterial from viral pneumonia, does not detect other conditions, and is not intended for clinical use.

> **Disclaimer.** PulmoScan AI is an educational prototype, not a medical device. It has not been clinically validated. Its output must never be used to diagnose, treat or rule out any condition and does not replace evaluation by a qualified healthcare professional.

2. Background and Related Work

2.1 Convolutional neural networks

A CNN is a neural network designed for grid-like data such as images. Instead of connecting every pixel to every neuron, it slides small filters over the image. Three properties make it well suited to image classification: *local connectivity* (each neuron looks at a small patch, matching the local nature of image features), *weight sharing* (the same filter is reused at every position, which greatly reduces parameters and detects a pattern wherever it occurs), and *hierarchical features* (stacked layers combine edges into textures and textures into larger structures). The filters are learned from labelled examples rather than hand-designed.

2.2 Deep learning for chest radiography

Kermany et al. (2018) released the labelled chest X-ray dataset used in this project and showed that transfer learning could classify paediatric pneumonia with high accuracy. Rajpurkar et al. (2017) trained CheXNet, a 121-layer DenseNet, on the much larger ChestX-ray14 dataset. These works rely on large pretrained networks and GPUs. This project deliberately takes a different route: a small network designed and trained from scratch, so that every component can be understood and explained, and so that it trains and runs on ordinary CPU hardware.

2.3 Explainability with Grad-CAM

Selvaraju et al. (2017) introduced Gradient-weighted Class Activation Mapping (Grad-CAM). It uses the gradients of a class score with respect to the feature maps of the last convolutional layer to produce a coarse heatmap of the regions that influenced the prediction. In healthcare applications such explanations help a user judge whether a prediction deserves trust, and they help the developer detect when a model is relying on irrelevant cues.

3. Dataset and Preprocessing

3.1 Dataset

The project uses *Chest X-Ray Images (Pneumonia)* by Kermany, Zhang and Goldbaum (2018), published on Mendeley Data under a CC BY 4.0 licence. It contains 5,856 chest X-rays of patients aged one to five from Guangzhou Women and Children's Medical Center, each labelled Normal or Pneumonia by expert physicians.

3.2 Data splitting

The dataset ships with a training folder and an official test folder. The widely used Kaggle copy also has a validation folder of only 16 images, which is far too small to guide training reliably. A new validation split was therefore created from the training pool, and the official test folder was left completely untouched for the final evaluation.

The split is *patient-grouped and stratified*. Filenames contain a patient identifier, and one patient can have several X-rays. If images of the same patient appeared in both training and validation, the validation score would be inflated, a form of data leakage. scikit-learn's `StratifiedGroupKFold` was used so that all images of a patient stay on the same side while the class ratio is preserved. The training pool contains 2,861 distinct patients.

Split	Normal	Pneumonia	Total	Purpose
Training	1,156	3,328	4,484	Learning the weights
Validation	193	555	748	Early stopping, learning-rate schedule
Test (official)	234	390	624	Final evaluation only

Table 1. Dataset splits used in this project.

3.3 Class imbalance

The training set has roughly three pneumonia images for every normal image. Left unaddressed, a model can score well simply by favouring the majority class. Balanced class weights were applied during training, so that an error on a Normal image counts 1.94 times and an error on a Pneumonia image 0.67 times in the loss.

3.4 Preprocessing

- **Grayscale conversion.** X-rays carry no colour information, so a single channel is sufficient.
- **Resize to 160 x 160 pixels** with bilinear interpolation. Pneumonia appears as relatively large hazy regions, so this resolution retains the relevant detail while allowing training on a CPU.
- **Normalisation** of pixel values from 0-255 to 0-1. This is implemented as a `Rescaling` layer inside the model, so it cannot be omitted at inference time.

The same preprocessing function (`backend/utils/preprocessing.py`) is used both to build the training data and to process user uploads in the API. Sharing one implementation guarantees that the model sees uploads exactly as it saw training images.

3.5 Data augmentation

To reduce overfitting, each training image is randomly transformed every epoch: random rotation (about +/-10 degrees); random shift (up to 8% horizontally and vertically); random zoom (up to 10%); random brightness (+/-10%); random contrast (+/-15%). Horizontal flipping was deliberately not used, because human anatomy is not symmetric (the heart lies on the left) and a mirrored X-ray is anatomically incorrect. Augmentation is applied only to the training set.

4. Model Architecture

The network, named PulmoScanCNN, consists of four convolutional blocks followed by a small classification head. Each block applies a 3 x 3 convolution, batch normalisation, a ReLU activation and 2 x 2 max-pooling. The number of filters doubles in each block (16, 32, 64, 128) while the spatial size halves (160, 80, 40, 20, 10).

Stage	Layers	Output shape	Parameters
Input	Grayscale image	160 x 160 x 1	0
Normalisation	Rescaling (1/255)	160 x 160 x 1	0
Conv block 1	Conv 3x3 (16 filters), BatchNorm, ReLU, MaxPool 2x2	80 x 80 x 16	208
Conv block 2	Conv 3x3 (32 filters), BatchNorm, ReLU, MaxPool 2x2	40 x 40 x 32	4,736
Conv block 3	Conv 3x3 (64 filters), BatchNorm, ReLU, MaxPool 2x2	20 x 20 x 64	18,688
Conv block 4	Conv 3x3 (128 filters), BatchNorm, ReLU, MaxPool 2x2	10 x 10 x 128	74,240
Pooling	Global average pooling	128	0
Regularisation	Dropout (0.4)	128	0
Dense	Fully connected, 64 units, ReLU	64	8,256
Regularisation	Dropout (0.3)	64	0
Output	Fully connected, 1 unit, sigmoid	1	65
Total			**106,193**

Table 2. PulmoScanCNN architecture (105,713 trainable parameters; saved model size 1.37 MB).

4.1 Design decisions

- **Batch normalisation** after every convolution keeps activations well-scaled, which stabilises and speeds up training. Because it adds its own shift, the convolution layers before it have no bias term.
- **Max-pooling** halves the resolution, reducing computation and enlarging the area of the original image each deeper neuron can see.
- **Global average pooling** instead of flattening: flattening the final 10 x 10 x 128 feature maps would give 12,800 values and a dense layer of about 820,000 weights, several times the size of the rest of the network. Averaging each map to one number gives 128 values and far less overfitting.
- **Dropout** (40% and 30%) randomly disables units during training so the network cannot depend on any single feature.
- **Sigmoid output.** One neuron outputs P(pneumonia); P(normal) is one minus that. For two classes this is equivalent to a two-way softmax with fewer parameters.
- **Compact size.** With about 4,500 training images, a small network is less prone to overfitting. It also had to train on a laptop CPU and run on free-tier hosting.

5. Training

Setting	Value
Framework	TensorFlow / Keras 2.21.0
Loss function	Binary cross-entropy
Optimizer	Adam, initial learning rate 0.001
Learning-rate schedule	ReduceLROnPlateau: halved after 3 epochs without improvement in validation loss
Batch size	32
Maximum epochs	25
Early stopping	Validation loss, patience 5, best weights restored
Class weights	Normal 1.94, Pneumonia 0.67
Batch-norm momentum	0.9
Hardware	Laptop CPU (Intel Core i5-1235U), no GPU

Table 3. Training configuration.

Training stopped automatically after 13 epochs, taking 30 minutes. The best validation loss occurred at epoch 8, and those weights were saved. At that epoch the training accuracy was 92.5% (measured with augmentation and dropout active, which makes it lower than it would be on clean images) and the validation accuracy was 95.2%.

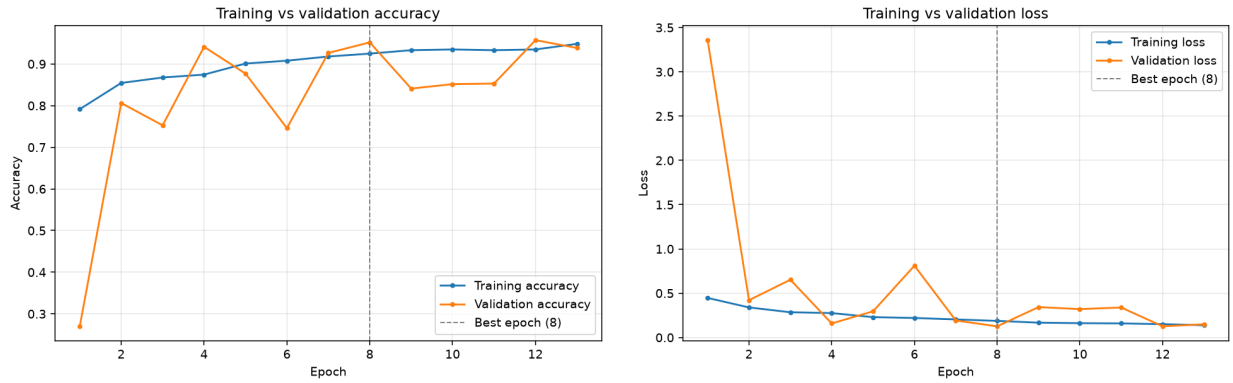


Figure 1. Training and validation accuracy (left) and loss (right) per epoch. The dashed line marks the best epoch.

5.1 Observations

The training curves rise smoothly, while the validation curves fluctuate from epoch to epoch. This behaviour is caused by batch normalisation: during training it normalises using the statistics of the current batch, but during validation it uses moving averages collected over time. When those averages lag behind the current weights, the network's outputs shift, and with a fixed 0.5 threshold a small shift changes many predictions. The validation ROC-AUC, which does not depend on the threshold, stayed high throughout.

5.2 Two training runs

The model was trained twice. In the first run, with the Keras default batch-norm momentum of 0.99, the validation accuracy stayed at the majority-class level for three epochs and collapsed again later in training. The instability was traced to slow moving averages. The momentum was reduced to 0.9 so that they track the weights more quickly, and the model was retrained. The decision was made from the validation curves. Both results are reported for transparency: the first run scored 72.9% on the test set and the final run scored 78.8%.

5.3 Hardware constraint

No GPU was available. An earlier design with a 224 x 224 input and two convolutions per block was too slow to train on the laptop CPU, so the input was reduced to 160 x 160 with one convolution per block. This is a real limitation on the accuracy achieved and is the first item under future work.

6. Results and Evaluation

All results in this chapter were measured on the official test set of 624 images, which was not used for training, for early stopping or for any design decision before the final run. The decision threshold is 0.5.

Metric	Value
Test accuracy	**78.8%**
ROC-AUC	**0.930**
Sensitivity (pneumonia recall)	98.5%
Specificity (normal recall)	46.2%
Macro-average F1-score	73.7%
Validation accuracy / ROC-AUC	95.2% / 0.988

Table 4. Headline results.

Class	Precision	Recall	F1-score	Support
Normal	94.7%	46.2%	62.1%	234
Pneumonia	75.3%	98.5%	85.3%	390
Macro average	85.0%	72.3%	73.7%	624
Weighted average	82.6%	78.8%	76.6%	624

Table 5. Per-class precision, recall and F1-score on the test set.

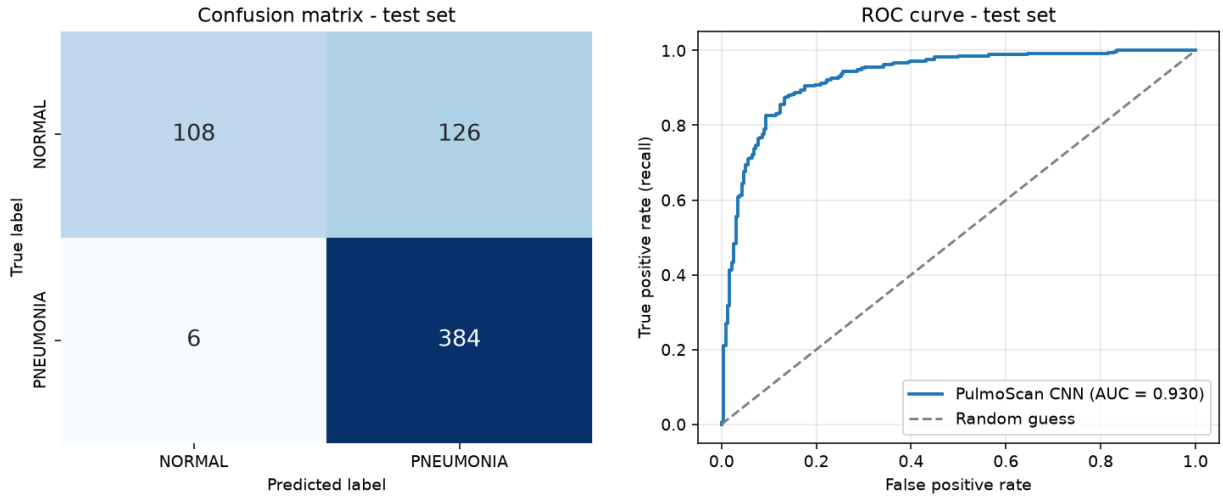


Figure 2. Confusion matrix (left) and ROC curve (right) on the test set.

6.1 Interpretation

The confusion matrix shows 384 true positives, 108 true negatives, 126 false positives and 6 false negatives. The model is therefore very sensitive: it identifies 384 of the 390 pneumonia X-rays and misses only 6. Its weakness is specificity: 126 of the 234 normal X-rays are incorrectly flagged as pneumonia. In a screening context a false negative is the more dangerous error, so the direction of the bias is the safer one, but the false-alarm rate is too high for the model to be useful in practice.

6.2 The validation-test gap

Validation accuracy is 95.2%, while test accuracy is 78.8%. On the validation set the errors are balanced (18 false positives and 18 false negatives among 748 images), so the model is not biased on data that resembles its training data. The gap is explained by three factors:

- **Distribution shift.** The official test images differ from the training pool. The model's scores for normal test images are shifted upwards, so many cross the 0.5 threshold. This gap is well known for this dataset and affects small networks trained from scratch in particular.
- **The threshold, not the ranking.** The test ROC-AUC of 0.930 shows that the model still ranks pneumonia images above normal ones well. A different threshold would give a much better balance on the test set.
- **Class balance.** The test set has a higher share of normal images (38%) than the training set (26%), so errors on normal images reduce accuracy more.

The threshold was deliberately not tuned on the test set. Choosing a threshold using test data would make the reported test score an optimistic, unreliable estimate. The result is reported as measured.

7. Explainability with Grad-CAM

Every prediction is accompanied by a Grad-CAM heatmap. The implementation (`backend/utils/gradcam.py`) follows these steps:

- Run the image through the network and keep the feature maps of the last convolutional block (20 x 20 x 128).
- Compute the gradient of the class score with respect to those feature maps. The score is p for Pneumonia and 1 - p for Normal, where p is the sigmoid output.
- Average each gradient map over space to obtain one importance weight per feature map.
- Form the weighted sum of the feature maps and apply ReLU, keeping only positive influence.
- Normalise to the range 0-1, upscale to the image size and blend it over the X-ray with a colour map.

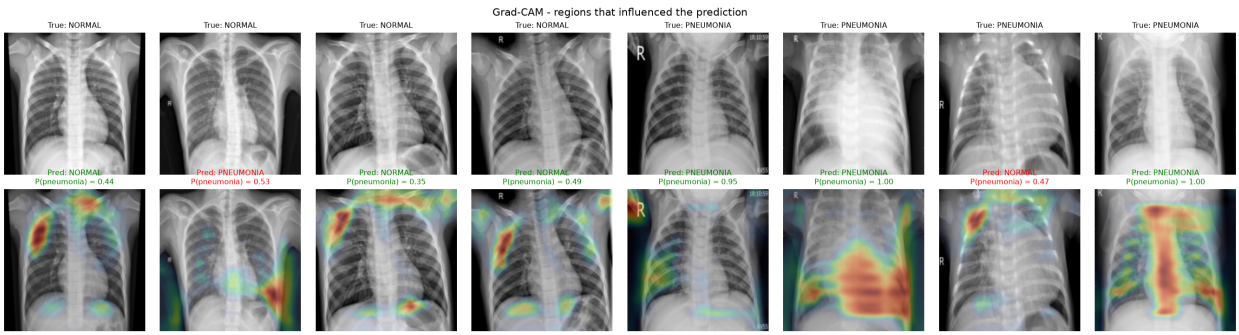


Figure 3. Grad-CAM for eight test images. Top: original with true label. Bottom: heatmap with the prediction (green = correct, red = incorrect).

7.1 What the heatmaps reveal

For the confident pneumonia predictions the model attends to the central and lower lung fields, which is plausible. The heatmaps also expose weaknesses. In one image the strongest activation lies on the letter "R" marker in the corner of the film rather than on the lungs, which indicates that the model has partly learned a shortcut from image artefacts. For several normal images the attention falls on the shoulders and image borders. These observations are exactly what explainability is for: they show that a correct label does not always come from correct reasoning, and they support the conclusion that the model is not reliable enough for real use.

A Grad-CAM heatmap shows which regions influenced the model's output. It is coarse and it is not a map of where disease is located.

8. System Design and Implementation

8.1 Architecture

The system has three parts: an offline machine learning pipeline, a model server and a web frontend. The frontend and the model server are deployed separately, because static hosting platforms cannot keep a Python process with TensorFlow loaded in memory.

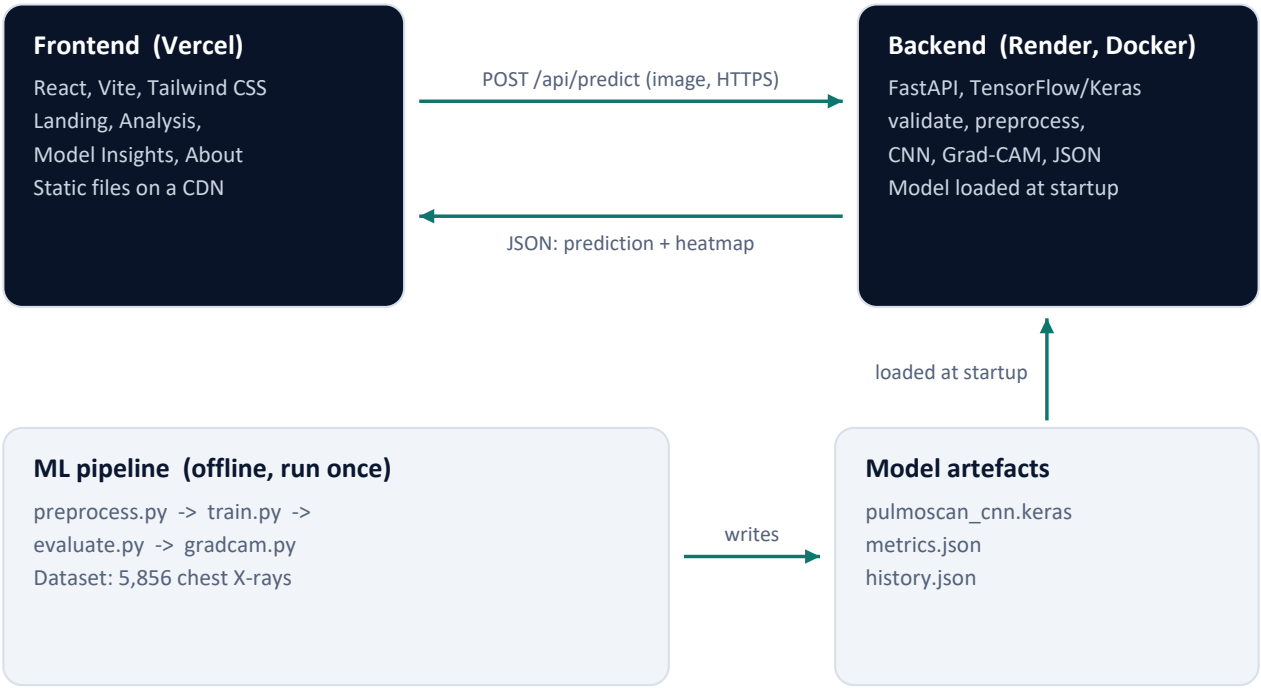


Figure 4. System architecture.

Layer	Technologies
Machine learning	Python, TensorFlow/Keras, scikit-learn, NumPy, Pillow, Matplotlib, Seaborn
Backend	FastAPI, Uvicorn, Pydantic, Docker
Frontend	React, Vite, Tailwind CSS, Framer Motion, Recharts, Lucide icons
Hosting	Vercel (frontend), Render (backend), GitHub (source code)

Table 6. Technology stack.

8.2 Machine learning pipeline

Script	Responsibility
`ml/preprocess.py`	Downloads and extracts the dataset, builds the patient-grouped split, resizes images and caches them as arrays
`ml/train.py`	Defines the CNN, applies augmentation and class weights, trains with early stopping, saves the model and training history
`ml/evaluate.py`	Computes all test metrics, draws the confusion matrix and ROC curve, writes `metrics.json`
`ml/gradcam.py`	Produces Grad-CAM figures for the report

8.3 Backend API

Endpoint	Method	Purpose
`/api/health`	GET	Reports whether the server is running and the model is loaded
`/api/predict`	POST	Accepts an image upload; returns the class, confidence, both probabilities and Grad-CAM images
`/api/model/insights`	GET	Returns the architecture, dataset details and measured metrics
`/docs`	GET	Interactive API documentation generated by FastAPI

The model is loaded once when the server starts and is kept in memory, so individual requests are fast. Uploads are validated on the server: a 10 MB size limit, verification with Pillow that the file is a genuine JPEG or PNG regardless of its name, and a minimum size of 100 pixels. An additional check warns the user when an upload contains a lot of colour, which is unusual for an X-ray. Uploaded images are processed in memory and are not stored.

Cross-Origin Resource Sharing (CORS) is restricted to an explicit list of origins: local development addresses and the deployed frontend. Browser requests from any other website are refused.

8.4 Frontend

Page	Contents
Landing	Product introduction, how it works, features, technology, live headline metrics
AI Analysis	Drag-and-drop upload with validation, preview, sample images, loading state, prediction, confidence gauge, probability breakdown, Grad-CAM viewer
Model Insights	Measured metrics, training curves, confusion matrix, per-class scores, ROC curve, dataset and architecture
About	Pneumonia, the importance of early evaluation, why CNNs suit images, how the model works, limitations

The Model Insights page contains no hard-coded numbers. It reads `metrics.json`, written by the evaluation script, through the API. If the model is retrained, the page updates automatically. The interface is responsive, provides skeleton loaders and toast notifications, and displays the medical disclaimer on every page and with every result.

9. Deployment

Component	Host	Address
Frontend	Vercel (static site)	https://pulmoscan-ai-topaz.vercel.app
Backend API	Render (Docker, free instance)	https://pulmoscan-api-8wdv.onrender.com
Source code	GitHub	https://github.com/Tanisha-dey18/pulmoscan-ai.

The frontend learns the address of the API from the build-time environment variable `VITE_API_URL`. In development this variable is empty and the Vite development server forwards API requests to the local backend, so the application code contains no hard-coded addresses. Configuration for Netlify (`netlify.toml`) is also included in the repository.

9.1 Problems encountered and solutions

Problem	Cause	Solution
Backend host unavailable	Hugging Face Spaces, the first choice, now requires a paid plan for Docker Spaces	Deployed the same Docker image on Render
Memory limit	The API uses about 420 MB with TensorFlow loaded; the free instance has 512 MB	Limited TensorFlow to one thread and reduced memory arenas in the Dockerfile
Frontend host blocked	The Netlify account had no deploy credits remaining	Deployed on Vercel; Netlify configuration kept in the repository
"Model server unreachable"	The API's CORS list did not include the deployed frontend's address	Added the frontend origin to the allowed list
404 on page refresh	The host looked for a file named after the route	Added a rewrite rule sending every path to index.html

Table 7. Deployment problems and their solutions.

9.2 Testing

- **API.** Health, prediction and insights endpoints return correct responses; non-image files and empty uploads are rejected with clear messages; requests from an unlisted origin are refused.
- **Live site.** A sample X-ray analysed on the deployed site returned the same prediction as the local system, with the Grad-CAM overlay; the Model Insights page displays the measured metrics; all routes load directly.
- **Responsiveness.** The pages were checked at phone width with no horizontal scrolling.
- **Performance.** A prediction takes about half a second locally and 3.5 to 5 seconds on the free server. The free server sleeps after 15 minutes without traffic, so the first request afterwards takes about a minute.

10. Limitations and Future Work

10.1 Limitations

- **Not clinically validated.** The system has never been tested in a clinical setting or reviewed by a regulator.
- **Low specificity.** Only 46.2% of normal test X-rays are recognised as normal; false alarms are frequent.

- **Narrow data.** All images are paediatric X-rays from a single hospital. Performance on adults or other equipment is unknown.
- **Two classes only.** The model cannot recognise other conditions and cannot tell whether an image is a chest X-ray at all.
- **Uncalibrated confidence.** The confidence shown is the raw sigmoid output, not a calibrated probability.
- **Shortcut learning.** Grad-CAM shows the model sometimes attends to film markers and image borders.
- **Single run.** Results come from one training run on one split, without cross-validation or confidence intervals.
- **Limited hardware.** Training on a CPU restricted the input resolution and network depth.

10.2 Future work

- Train on a GPU at higher resolution with a deeper network, or use transfer learning (DenseNet, EfficientNet).
- Train on larger, multi-hospital datasets such as NIH ChestX-ray14 or CheXpert, and validate on external data.
- Reduce shortcut learning by cropping or masking film markers and using stronger augmentation.
- Choose the decision threshold on validation data that resembles deployment data, and calibrate the probabilities.
- Add an out-of-distribution check that rejects images which are not chest X-rays.
- Extend to bacterial versus viral pneumonia, and use k-fold cross-validation to report confidence intervals.

11. Conclusion

This project delivered a complete deep learning application: a reproducible training pipeline, a CNN designed and trained from scratch, an evaluation on untouched test data, Grad-CAM explanations, a validated REST API, a responsive web interface, and a public deployment. The model reaches 78.8% accuracy and 0.930 ROC-AUC on the official test set, with very high sensitivity (98.5%) and low specificity (46.2%).

The most valuable lessons were about evaluation rather than modelling. A 16-image validation folder, patient-level leakage, class imbalance and distribution shift can each make a model look better than it is. Addressing them, and reporting the weaker test result unchanged, gives a result that can be defended. The Grad-CAM analysis reinforced this by showing that the model sometimes reaches a correct answer for the wrong reason. The system demonstrates the full engineering path from raw data to a deployed, explainable application, and it documents clearly what would be needed to make the model itself stronger.

References

[1] Kermany, D. S., Goldbaum, M., Cai, W., et al. (2018). Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5), 1122-1131.

[2] Kermany, D., Zhang, K., Goldbaum, M. (2018). Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images for Classification. *Mendeley Data*, v2. doi:10.17632/rscbjbr9sj.2

[3] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.

[4] Rajpurkar, P., Irvin, J., Zhu, K., et al. (2017). CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning. *arXiv:1711.05225*.

[5] Ioffe, S., Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of ICML*.

[6] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929-1958.

[7] Kingma, D. P., Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of ICLR*.

[8] Lin, M., Chen, Q., Yan, S. (2014). Network in network. *Proceedings of ICLR*. (Global average pooling.)

[9] World Health Organization. Pneumonia in children: fact sheet.

[10] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *Proceedings of OSDI*.